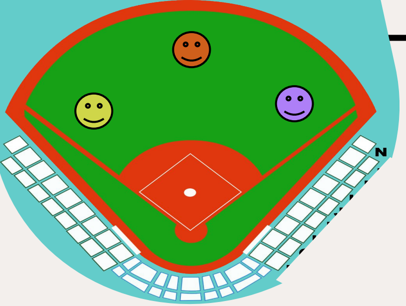
MLB
DATA ANALYSIS

Group 11

David Sweasey, John Farrell, and Morgan Rivera

Business Goal

- **Goal** is to predict the best offensive MLB team for the 2025 season.
- **Datasets** we chose were from Rotowire for the 2023 and 2024 MLB seasons. These datasets list offensive batting information about each player.
- **Unit of Observation:** Teams
- **Unit of Analysis:** Teams



Team Composition

DAVID

Team Speed

- **Stolen Bases (SB):** require fast players to obtain
- **On-Base Percentage (OBP):** can be higher for fast players
- **Team speed:** can result in more runs scored = greater offense!

JOHN

Performance Based on Position

- **Position (Pos):** the position played by the batter
- **Games (G):** the number of games played
- **At bats (AB):** the number of times the player appeared at the plate

MORGAN

Teams Scoring Potential

- **Batting Average (AVG):** frequency on base with successfully hit
- **Home Runs (HR):** team's power-hitting ability
- **Runs Batted In (RBI):** effectiveness in plays

Timeline

3/30 Create Individual EDA

3/31 Plan Analysis Sketch

Easy to do, save time for decision tree in case of any issues

4/2 Create Offensive Skill Metric

4/4 Find Top 3 Players From Each Team And W/L Records For 2024 Games

Finished sooner than anticipated (-2 days)

Most likely to take bulk of time and may need to revise

4/6 Create Decision Tree

4/11 Revise Decision Tree

Struggled with Decision Tree Accuracy (+5 days)

Built in buffer for any delays

4/12 Work On Final Analysis and Presentation

4/14 Finalize Presentation

Started later than anticipated (+1 day)

Exploratory Data Analysis

Player and Team Speed

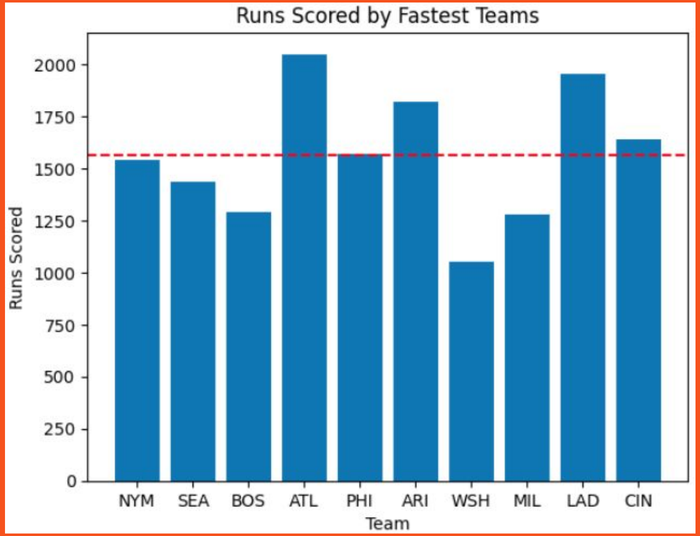
- Analyzed relationship between Stolen Bases (SB), On-Base Percentage (OBP), and Runs scored (R)
- Typically reflects player speed
- Team speed determined by total SB and average OBP
- Fastest teams:
    - Above average in OBP
    - Top 10 in SB

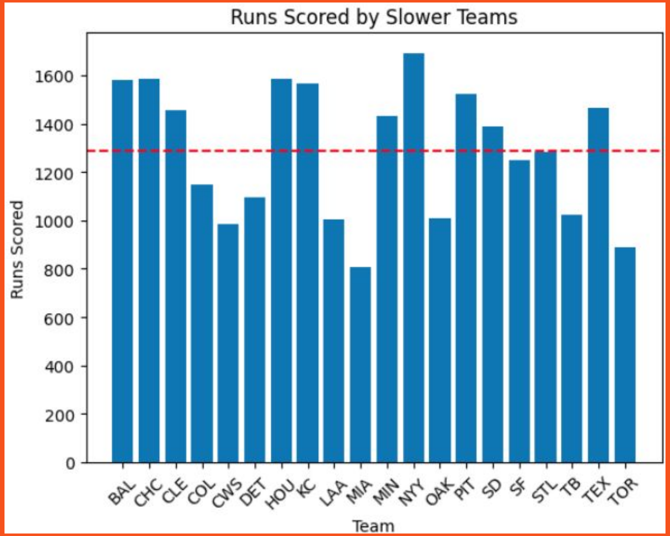
	Team	OBP	SB
0	CIN	0.315514	378
1	LAD	0.323688	323
2	MIL	0.319720	315
3	WSH	0.316357	286
4	ARI	0.326571	280
5	PHI	0.322370	275
6	ATL	0.317889	245
7	BOS	0.312914	243
8	SEA	0.316867	235
9	NYM	0.313871	234

Exploratory Data Analysis

Player and Team Speed

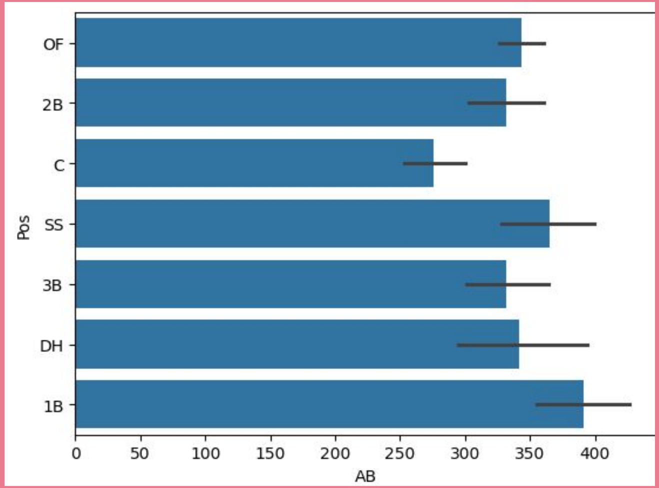
- **Fastest teams *tend* to score more runs**
- **Slower teams *tend* to score less runs**

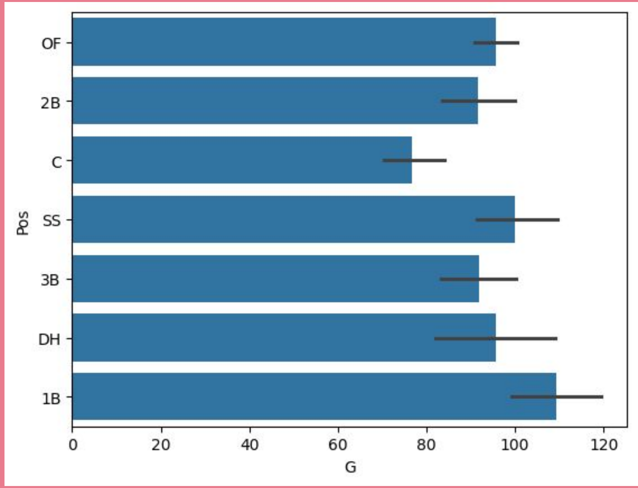




Exploratory Data Analysis

- Analyzed the relationships between games, at bats, and positions
    - The games and at bats are highly correlated with each other reflected in the little difference between and and at bats per position





Exploratory Data Analysis

- **Analyzed the distribution of positions for each team and found the average number of players for each position on a team**

```
OF 10.766666666666667
2B 3.9
C 4.466666666666667
SS 3.2
3B 3.566666666666667
DH 1.3666666666666667
1B 3.033333333333333
0.006201505661010742
```

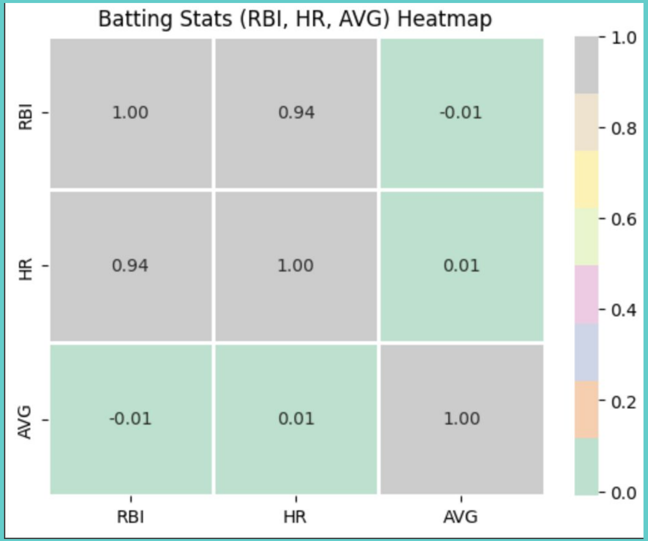
Pos	1B	2B	3B	C	DH	OF	SS
Team							
ARI	4	2	3	7	2	12	5
ATL	2	9	5	4	2	12	2
BAL	4	6	6	4	2	9	2
BOS	3	8	2	6	2	10	4
CHC	4	4	6	7	1	9	2
CIN	8	2	4	5	1	15	2
CLE	2	2	3	6	1	14	5

Exploratory Data Analysis

Teams Scoring Potential

Correlation of RBI, HR, AVG

- **Strong correlation (0.94) between HR and RBI - As RBI *increases*, HR tends to *increase***
- **No correlation (+/-0.01) between AVG and HR as well as AVG and RBI**

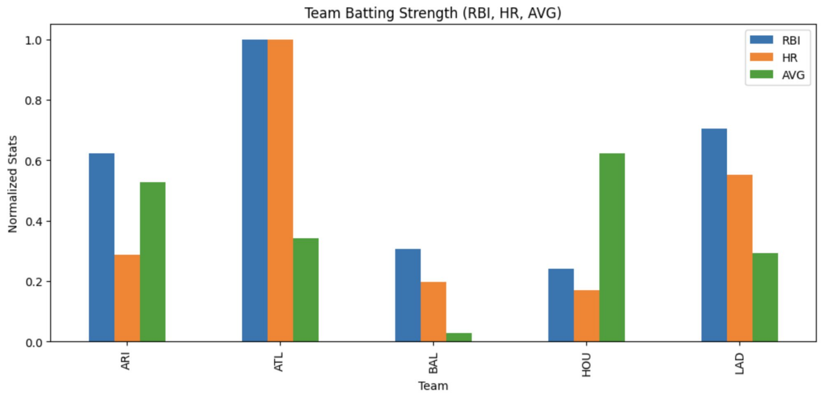


Exploratory Data Analysis

Teams Scoring Potential

Comparing Top Teams RBI, HR, and AVG

- **High AVG and Low HR/RBI**
  - **ARI and HOU have frequent hits but not translating them into RBI or HR effectively.**
- **Low/Mid AVG and High HR/RBI**
  - **ATL, BAL, and LAD with high RBI and HR, show strong offense despite fewer total hits.**



Wrangling

- Split original merged data based on year

```
stats_24 = df_merged[df_merged["Year"] == 2024]
stats_23 = df_merged[df_merged["Year"] == 2023]
```

- Grouped by team and aggregated player stats = Team Dataframe
- Calculated *BAT_SCORE* for all players
  - (Player runs) / (Team runs) +
  - (Player RBI) / (Team RBI) +
  - OPS

NYY	Aaron Judge	1.478904
LAD	Shohei Ohtani	1.348448
TOR	Vladimir Guerrero	1.339943
KC	Bobby Witt	1.283430
NYY	Juan Soto	1.272442

- Kept only top 3 players in *BAT_SCORE* from each team
- Added these players to the Team Dataframe
- Recombined 2023 and 2024 dataframes

	Team	P1_NAME	P1_SCORE	P2_NAME	P2_SCORE	P3_NAME	P3_SCORE
0	ARI	Ketel Marte	1.140844	Joc Pederson	1.047980	Eugenio Suarez	1.000315
1	ATL	Marcell Ozuna	1.179116	Matt Olson	1.014975	Jorge Soler	0.967840
2	BAL	Gunnar Henderson	1.147062	Anthony Santander	1.047982	Colton Cowser	0.944774
3	BOS	Rafael Devers	1.105563	Jarren Duran	1.090154	Tyler O'Neill	1.033574
4	CHC	Seiya Suzuki	1.050704	Ian Happ	1.023254	Michael Busch	0.965058

Wrangling

Why ?

- Counteract outliers in RBI, R, and AVG
- Includes <u>Star Players</u> into our analysis
- Leverage player stats and team stats

Target Value

- As our datasets had no good indicator of a good team vs a bad team except for batting average which does not tell the full story
- We added a new dataset containing the win loss ratio of each team to use in our model
    - The win loss ratio is used as the Y value in our model

	Rk	Tm	W-L%
0	9.0	Arizona Diamondbacks	0.549
1	8.0	Atlanta Braves	0.549
2	7.0	Baltimore Orioles	0.562
3	18.0	Boston Red Sox	0.500
4	16.0	Chicago Cubs	0.512
5	22.0	Cincinnati Reds	0.475
6	6.0	Cleveland Guardians	0.571

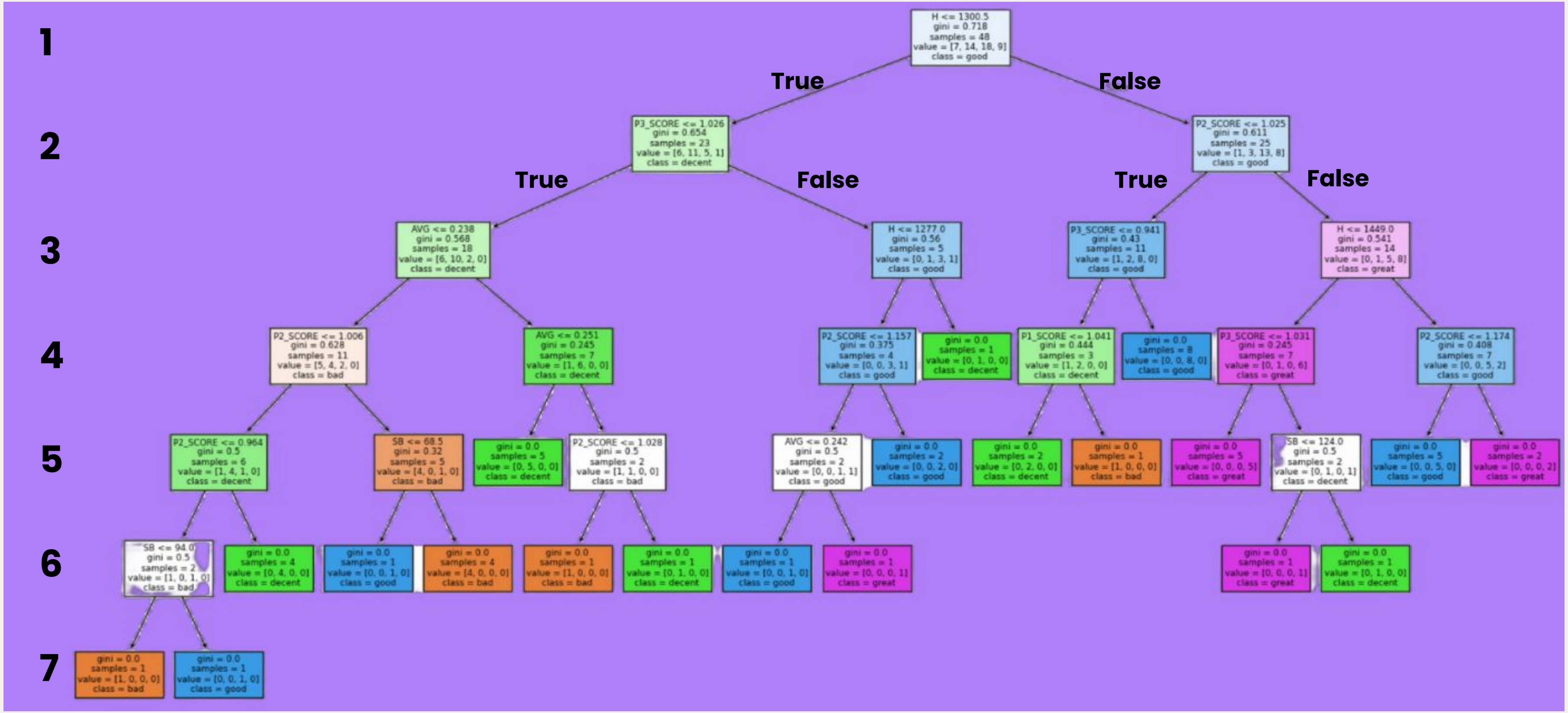
```
# How many "bins" to discretize the y into
bins = [0, 0.4, 0.5, 0.57, 1]
labels = ['bad', 'decent', 'good', 'great']
y = pd.cut(winloss['W-L%'], bins=bins, labels=labels)
```

Modeling

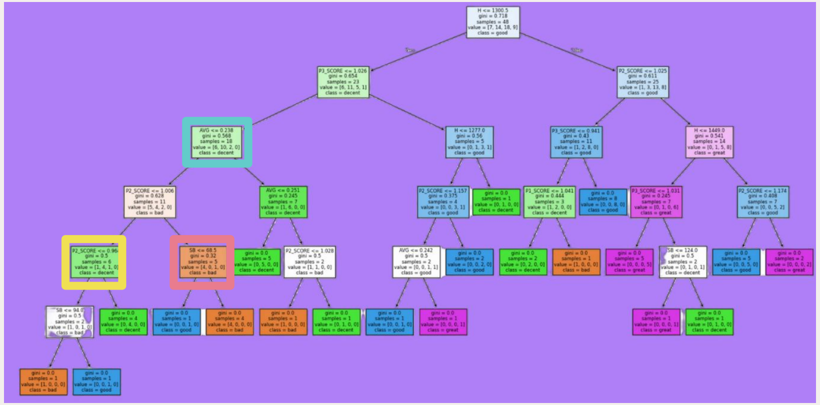
Why a Decision Tree?

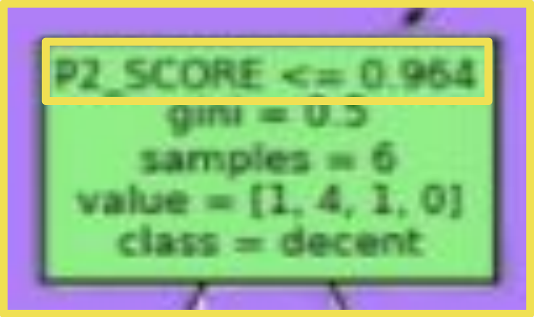
- Simple supervised model
- Great for classification with numerical data
- Easily interpretable
- Easy to construct
- Resistant to outliers
- Provides additional information on feature importance

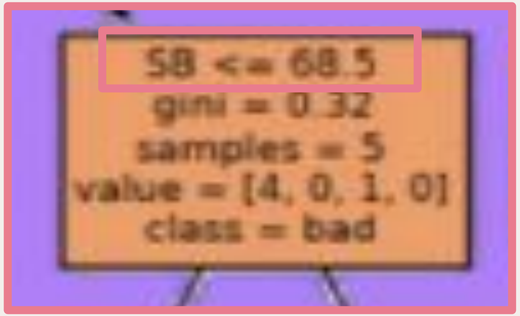
Modeling

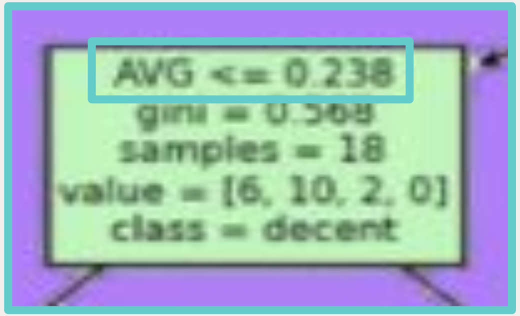

1
2
3
4
5
6
7
True
False
True
False
True
False
H <= 1300.5 gini = 0.718 samples = 48 value = [7, 14, 18, 9] class = good
P3_SCORE <= 1.026 gini = 0.654 samples = 23 value = [6, 11, 5, 1] class = decent
P2_SCORE <= 1.025 gini = 0.611 samples = 25 value = [1, 3, 13, 8] class = good
AVG <= 0.238 gini = 0.568 samples = 18 value = [6, 10, 2, 0] class = decent
H <= 1277.0 gini = 0.56 samples = 5 value = [0, 1, 3, 1] class = good
P3_SCORE <= 0.941 gini = 0.43 samples = 11 value = [1, 2, 8, 0] class = good
H <= 1449.0 gini = 0.541 samples = 14 value = [0, 1, 5, 8] class = great
P2_SCORE <= 1.006 gini = 0.628 samples = 11 value = [5, 4, 2, 0] class = bad
AVG <= 0.251 gini = 0.245 samples = 7 value = [1, 6, 0, 0] class = decent
P2_SCORE <= 1.157 gini = 0.375 samples = 4 value = [0, 0, 3, 1] class = good
gini = 0.0 samples = 1 value = [0, 1, 0, 0] class = decent
P1_SCORE <= 1.041 gini = 0.444 samples = 3 value = [1, 2, 0, 0] class = decent
gini = 0.0 samples = 8 value = [0, 0, 8, 0] class = good
P3_SCORE <= 1.031 gini = 0.245 samples = 7 value = [0, 1, 0, 6] class = great
P2_SCORE <= 1.174 gini = 0.408 samples = 7 value = [0, 0, 5, 2] class = good
P2_SCORE <= 0.964 gini = 0.5 samples = 6 value = [1, 4, 1, 0] class = decent
SB <= 68.5 gini = 0.32 samples = 5 value = [4, 0, 1, 0] class = bad
gini = 0.0 samples = 5 value = [0, 5, 0, 0] class = decent
P2_SCORE <= 1.028 gini = 0.5 samples = 2 value = [1, 1, 0, 0] class = bad
AVG <= 0.242 gini = 0.5 samples = 2 value = [0, 0, 1, 1] class = good
gini = 0.0 samples = 2 value = [0, 0, 2, 0] class = good
gini = 0.0 samples = 2 value = [0, 2, 0, 0] class = decent
gini = 0.0 samples = 1 value = [1, 0, 0, 0] class = bad
gini = 0.0 samples = 5 value = [0, 0, 0, 5] class = great
SB <= 124.0 gini = 0.5 samples = 2 value = [0, 1, 0, 1] class = decent
gini = 0.0 samples = 5 value = [0, 0, 5, 0] class = good
gini = 0.0 samples = 2 value = [0, 0, 0, 2] class = great
SB <= 94.0 gini = 0.5 samples = 2 value = [1, 0, 1, 0] class = bad
gini = 0.0 samples = 4 value = [0, 4, 0, 0] class = decent
gini = 0.0 samples = 1 value = [0, 0, 1, 0] class = good
gini = 0.0 samples = 4 value = [4, 0, 0, 0] class = bad
gini = 0.0 samples = 1 value = [1, 0, 0, 0] class = bad
gini = 0.0 samples = 1 value = [0, 1, 0, 0] class = decent
gini = 0.0 samples = 1 value = [0, 0, 1, 0] class = good
gini = 0.0 samples = 1 value = [0, 0, 0, 1] class = great
gini = 0.0 samples = 1 value = [0, 0, 0, 1] class = great
gini = 0.0 samples = 1 value = [0, 1, 0, 0] class = decent
gini = 0.0 samples = 1 value = [1, 0, 0, 0] class = bad
gini = 0.0 samples = 1 value = [0, 0, 1, 0] class = good

Modeling




P2_SCORE <= 0.964
gini = 0.5
samples = 6
value = [1, 4, 1, 0]
class = decent

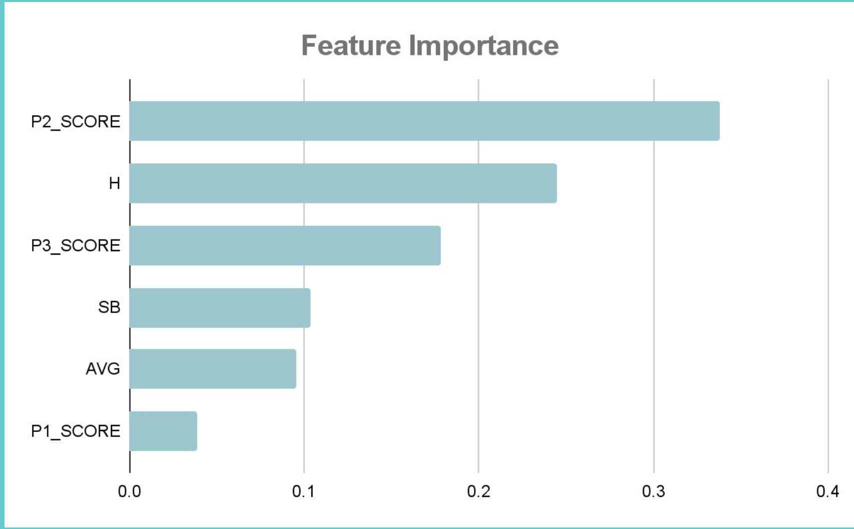

SB <= 68.5
gini = 0.32
samples = 5
value = [4, 0, 1, 0]
class = bad


AVG <= 0.238
gini = 0.568
samples = 18
value = [6, 10, 2, 0]
class = decent

Modeling

Evaluation - Feature Importance

1. **P2_SCORE** **0.338**
2. **H** (Hits) **0.245**
3. **P3_SCORE** **0.178**
4. **SB** (Stolen Bases) **0.104**
5. **AVG** (Batting Average) **0.096**
6. **P1_SCORE** **0.039**



Modeling

Evaluation

- **Accuracy: 0.5**
- **Mean-Squared Error: 0.75**

```
Accuracy: 0.5
Predictions: ['good' 'good' 'decent' 'good' 'good' 'decent' 'great' 'great' 'bad'
 'great' 'decent' 'good']
Mean Squared Error: 0.75
```

- **Not bad. Why?**
  - **Output/target value is a continuous value that is binned. Getting exact bin is tough**
  - **Our model is offense only**
  - **Teams get lucky, or play bad opponents**

Modeling

Results

- **We wrote a program to web crawl and retrieve a team's current roster**
- **Can be automatically updated as rosters change**
  - **More dynamic than datasets**
- **With a teams roster, used 2024 or 2023 dataset to aggregated team stats (similar to wrangling)**
- **Ran the team through the decision tree**

```
players = # roster from the webcrawler

new_team = build_team(players)
display(new_team)
prediction = tree.predict(new_team[features])
prediction[0]
```

AVG	OBP	SLG	OPS	P1_NAME	P1_SCORE	P2_NAME	P2_SCORE	P3_NAME	P3_SCORE
0.236417	0.299083	0.382917	0.682	Elly De La Cruz	1.138217	Tyler Stephenson	1.026987	Spencer Steer	1.022457

Modeling

Results

Team	Model Prediction	Current W/L
Cincinnati Reds (David's Favorite Team)	- bad	.500
Washington Nationals (Morgan's Favorite Team)	- decent	.375
LA Dodgers (World Series Champs 2024)	- great	.647
NY Yankees (World Series Runner Up 2024)	- good	.563
Philadelphia Phillies (ESPN Top 5 Team)	- great	.563
San Diego Padres (ESPN Top 5 Team)	- good	.813
Colorado Rockies (ESPN Worst Team)	- bad	.200
Chicago Cubs (Best Offensive Team Currently)	- great	.611

Bins: *bad (0 - 0.4)* *decent (0.4 - 0.5)* *good (0.5 - 0.57)* *great (0.57 - 1)*

In baseball, ~0.55 and above can usually get a team into the playoffs

Conclusion

- Predicts this season's notable teams and are fairly accurate so far
  - Doesn't necessarily find the team winning the World Series
  - Decision tree classifies teams into bucket based on W/L record
- May be use to find the most likely World Series champions, but would want to improve quality by adding defensive statistics, player's current team, etc.

Future Work

- Sport Analysts and Sport Bettors may use this model to:
  - Analyze more baseball statistics (MLB, college, etc.)
    - Add more years of data
    - Add defensive statistics
  - Can manipulate to be applicable to other sports
  - Expand to be more predictive and determine future statistics with higher accuracy

Limitations

- Dataset had no win-loss record of teams to determine team performance
    - Easily accessible and simple to add to help create model
- Dataset is exclusive to offensive statistics
    - Defensive statistics could paint a better picture and offer more insight on teams

Challenges

- Duplicate players in dataset from their statistics from 2023 and 2024
  - Split the dataset by year and then found the top players
  - Treated the teams by year as their own entity
- Formating of win-loss record not compatible with dataset
  - Had to reorganize to add to dataset and contribute to the analysis

Resources and References

- MLB 2023 Batting Statistics
  - https://www.rotowire.com/baseball/stats.php?season=2023
- MLB 2024 Batting Statistics
  - https://www.rotowire.com/baseball/stats.php?season=2024
- Exploratory Data Analysis - David Sweasey
  - https://colab.research.google.com/drive/1tGAg2UBY7LVCC2osmItC3e-09vHxQQNm?usp=sharing
- Exploratory Data Analysis - John Farrell
  - https://colab.research.google.com/drive/1RJu26Gz8Ssr8GL71I0MK25LJVypS1_N5?usp=sharing
- Exploratory Data Analysis - Morgan Rivera
  - https://colab.research.google.com/drive/1ZRVzzLvyzjzGBac9MXOcqH7N6y-7tBTy?usp=sharing
- Team Data Analysis
  - https://colab.research.google.com/drive/14V2cw4xgzu7wkupu7wEn-4VeBOS-m34j?usp=sharing
- Team Getter Data Analysis
  - https://colab.research.google.com/drive/13hakHWNvpBrRgKOBq4_9yKHYIOMzRu6n?usp=sharing
- GitHub
  - https://github.com/csc442-team11/MLB-Data-Analysis.git